

MULTI-STEP KNOWLEDGE EDITING AS CONTROLLED TARGET-WRITE DYNAMICS

SCHEDULES, PRESERVATION BARRIERS, AND RESPONSE-BASED CORESETS

Research working note

14 September 2026

ABSTRACT

We formulate repeated target-vector optimization and parameter writing as a controlled hybrid dynamical system. The controls allocate target-optimization steps, write scales, and the number of refinements within a request. The motivation is a reported Llama 3/AlphaEdit result: at layer 4, a short target optimization followed by a write and a full-budget target optimization followed by another write improves three editing metrics over one pass. We separate this observation from an unverified mechanism: refreshed targets might guide the editor to better endpoints while avoiding high preservation loss. We define that barrier relative to a declared path family, show that scalar speed alone cannot change an autonomous trajectory, and characterize the fixed write subspace. For inexpensive loss evaluation, we propose weighted medoids of finite update responses, separate vulnerable-example checks, and independent evaluation data. A finite-candidate approximation bound states exactly what this coreset controls. The citation audit covers direct iterative-editing precedents, target-through-write optimization, barrier and stability methods, and the backward and forward literature on coreset selection. This document specifies a research program; it reports no new LLM experiments or verified barrier bypass.

1 MOTIVATION AND THE CLAIM TO TEST

The motivating in-house report uses Llama 3, AlphaEdit, batches of 100 requests, and 10,000 edits in total. Both target optimization and parameter writing are at layer 4. For each incoming request batch, a first pass optimizes the target for fewer than the usual 25 steps and writes the update; a second pass reoptimizes the target on the changed model with the full configured budget and writes again. All three reported editing metrics improve over one pass. The exact short budget, scales, metric definitions, effect sizes, random seeds, and cache/history behavior still require inspection of the run. Here, “full” means the configured budget, not mathematical convergence.

Causal tracing measures recovery under a specified activation intervention. It does not solve the constrained optimization problem that chooses persistent weight updates. ROME and MEMIT provide the localization and editing context (Meng et al., 2022; 2023); localization need not predict which intervention is easiest to edit successfully (Hase et al., 2023). BLUE studies errors in distributing a downstream residual across layers (Li et al., 2025). These observations motivate testing a local layer-4 target and write, but do not establish that knowledge is stored exclusively there. Layer numbers and tracing protocols are also backbone dependent.

Our specific hypothesis is that *the timing of target recomputation changes the executable update direction and implicit regularization*. This may improve preservation and generalization at matched efficacy. It need not involve a loss barrier. The barrier interpretation becomes a separate empirical claim with a falsifiable definition in Section 4. This note develops a new target-refresh formulation; the repository’s separate `ode.tex` describes a flow based on a frozen raw proposal.

2 AN EDITABLE STATE AND ITS CONSTRAINED WRITE OPERATOR

Use outer index t for incoming request batches and inner index k , or time s , for refinements of one batch. Suppress t within an episode. Let $W_{\text{in}} \in \mathbb{R}^{d \times p}$ be the incoming layer-4 MLP output matrix,

and let W denote its provisional value. Other parameters remain fixed. Let $K \in \mathbb{R}^{p \times n}$ contain the input keys for the n current requests, $H(W) \in \mathbb{R}^{d \times n}$ their target-location activations, and $Z \in \mathbb{R}^{d \times n}$ the desired activations. The planned residual is $R = Z - H(W)$.

For fixed tokenized prompts, deterministic inference, and editing only this output matrix, its input keys do not change during the episode. Downstream representations and predictions do change. Implementations can average keys over contexts while measuring targets on different contexts; consequently $H(W + \Delta) - H(W) = \Delta K$ is an additional matching assumption, not an automatic identity for all extracted tensors.

Behavioral objectives. For current request i , let $E_i(W)$ be the declared target-token negative log likelihood. Write $E(W) = n^{-1} \sum_i E_i(W)$ for optimization, and define the individual success set

$$\mathcal{G}_\varepsilon = \{W : E_i(W) \leq \varepsilon_i \text{ for every current request } i\}. \quad (1)$$

An average loss cannot substitute for these individual requirements. The generic preservation objective uses a *fixed original model*:

$$D(W) = \mathbb{E}_{x \sim \mathcal{D}_{\text{ctrl}}} \text{KL}(p_{\text{orig}}(\cdot | x) \| p_W(\cdot | x)). \quad (2)$$

Here x includes fixed teacher-forced token prefixes and scoring positions. Keep the reference, vocabulary, and reduction unchanged throughout the sequential stream. An incoming-model KL would instead measure incremental drift and could hide cumulative damage. Preserve previously accepted desired edits with separate losses $R_j^{\text{old}}(W) \leq c_j$, $j \in \mathcal{J}_t$; the original teacher may disagree with those edits. Held-out paraphrases, neighborhood queries, and general-capability benchmarks are evaluation data.

Target optimization. Let $J(W, Z)$ be the loss under the target activation intervention. A convenient smooth model is

$$J(W, Z) = E_{\text{hook}}(W, Z) + \beta D_{\text{hook}}(W, Z) + \frac{\rho}{2} \|Z - H(W)\|_F^2. \quad (3)$$

Specify the reference used inside D_{hook} separately from Equation (2). The quadratic regularizer here is a modeling choice. Reproducing AlphaEdit requires its actual target optimizer, loss head, norm penalty, clamp, context construction, and reference initialization (Fang et al., 2025; AlphaEdit authors, 2025); Equation (3) is not a claim that its code uses this exact objective.

An exact projected quadratic model of the write. Let $P = UU^\top$ be an orthogonal projector with $U^\top U = I_r$, let $C \succeq 0$ be the incoming previous-key second moment, and let $\lambda > 0$. Define

$$\mathcal{A}(W, Z; C) = \arg \min_{\Delta = \Delta P} \left\{ \frac{1}{2} \|\Delta K - R\|_F^2 + \frac{1}{2} \text{tr}(\Delta C \Delta^\top) + \frac{\lambda}{2} \|\Delta\|_F^2 \right\}. \quad (4)$$

With $\bar{K} = U^\top K$ and $\bar{C} = U^\top C U$, the solution is

$$\boxed{\mathcal{A}(W, Z; C) = R A_C, \quad A_C = \bar{K}^\top (\bar{K} \bar{K}^\top + \bar{C} + \lambda I_r)^{-1} U^\top.} \quad (5)$$

Substitute $\Delta = B U^\top$ into Equation (4); the normal equations are $B(\bar{K} \bar{K}^\top + \bar{C} + \lambda I_r) = R \bar{K}^\top$. Positive definiteness gives a unique solution. This is the ideal orthogonal projection form of an AlphaEdit-style write, with explicit ridge strength. Projection thresholds, finite precision, and the deployed implementation need separate checks. The quadratic subproblem is convex: any proposed *nonconvex barrier concerns actual model losses or target optimization*, not a barrier inside this solve.

3 DISCRETE SCHEDULES AND A HYBRID ODE

Let $\text{Opt}_m(W, H(W))$ mean m steps of the declared target optimizer with W fixed and a fresh target initialization. A schedule $u = (N, \{m_k, \eta_k\}_{k=0}^{N-1})$ induces

$$\begin{aligned} Z_k &= \text{Opt}_{m_k}(W_k, H(W_k)), & R_k &= Z_k - H(W_k), \\ \Delta_k &= R_k A_C, & W_{k+1} &= W_k + \eta_k \Delta_k, \quad W_0 = W_{\text{in}}. \end{aligned} \quad (6)$$

The ordinary one-pass setting is $(N, m_0, \eta_0) = (1, m_{\text{full}}, 1)$. The reported experiment uses $N = 2$, $m_0 < m_{\text{full}}$, and $m_1 = m_{\text{full}}$; its scales remain to be verified.

For our controlled comparison, freeze C throughout inner refinement and register the accepted current keys once afterward, e.g. $C_{t+1} = C_t + K_t K_t^\top$ with the baseline’s normalization. Repeating an editor entry point may instead update C after every call. That changes the regularizer and is a separate experimental arm. Target caches must not silently reuse Z_0 for a pass intended to reoptimize it. Each candidate branch starts from an identical model, history, target-cache, optimizer, and random state.

Two controlled clocks. A continuous relaxation is

$$\dot{Z}(s) = -a(s)\nabla_Z J(W(s), Z(s)), \quad \dot{W}(s) = b(s)\mathcal{A}(W(s), Z(s); C), \quad a, b \geq 0. \quad (7)$$

In a target phase set $b = 0$; in a write phase set $a = 0$. One explicit Euler write with step mass η_k yields Equation (6). At a fresh target phase, reset $Z \leftarrow H(W)$ and reset optimizer state if the implementation does so. The faithful object is therefore a switched/hybrid system, with continuous phases and reset events. A simultaneous smooth flow is another algorithm. For plain gradient descent, a target horizon is approximately m_k times the learning rate; Adam needs its moment states for a more faithful representation. A finite, unit-scale write is not an exact ODE integration. Target-solver error and write-discretization error are distinct.

Proposition 1 (Rate control versus route control). *If F is a fixed autonomous vector field with unique trajectories, a flow $\dot{W} = b(s)F(W)$, $b \geq 0$, follows the same trajectory as $dW/dq = F(W)$ under $q(s) = \int_0^s b(v) dv$. If F is constant, the path is a ray.*

Proof. Compose the solution of $dW/dq = F(W)$ with $q(s)$ and apply the chain rule almost everywhere. Zero-rate intervals pause the trajectory. $\square$

Multiplying both clocks in Equation (7) by a common positive rate also only changes time, provided the same reset events occur. A different *relative* allocation of target and write effort, or a different reset schedule, can change the route. Optimizing Z and writing W generally do not commute: the write changes the model on which the next target is learned. This mechanism requires no change in the layer-4 input keys.

Proposition 2 (Fixed-operator endpoint equivalence). *For fixed K, U, C, λ , every schedule in Equation (6) ends in*

$$W_N = W_{\text{in}} + B_N A_C, \quad B_N = \sum_{k=0}^{N-1} \eta_k R_k. \quad (8)$$

An unrestricted single target $Z_{\text{eff}} = H(W_{\text{in}}) + B_N$ produces the same endpoint with a unit-scale write.

Proof. Sum the increments in Equation (6); then substitute Z_{eff} into Equation (5). $\square$

Thus refinement does not enlarge the affine write space $W_{\text{in}} + \{B A_C : B \in \mathbb{R}^{d \times n}\}$. Its possible advantage is finding a better effective target under optimizer, regularization, and compute limits. The unrestricted Z_{eff} need not satisfy the ordinary target clamp or be found by its optimizer. A direct optimization through the write is an essential comparator, especially given MetaKE (Liu et al., 2026b).

Repeated residual fitting is already a competing explanation. Suppose the target is frozen at Z_* and, additionally, the extraction conventions satisfy $H(W + \Delta) - H(W) = \Delta K$. Put $M = A_C K$. Then

$$R_{k+1} = R_k(I_n - \eta_k M), \quad W_N - W_{\text{in}} = R_0 \left[\sum_{k=0}^{N-1} (I_n - M)^k \right] A_C \quad (\eta_k = 1). \quad (9)$$

M is positive semidefinite with eigenvalues in $[0, 1)$, since the ridge and history term make the projected Gram inverse strictly regularized. Each positive-eigenvalue residual component contracts at unit scale. In a batch, the polynomial in M applies different gains to different request modes;

fixed-target repetition is generally *not* a scalar rescaling of the first delta. For one matched key it is a scalar filter. Therefore a gain from two writes need not demonstrate fresh-target feedback, let alone barrier avoidance. Equation (9) gives a specific control arm.

Extension to several layers. One can use $\dot{W}_\ell = b_\ell(s)\mathcal{A}_\ell(W, Z; C_\ell)$ with per-layer clocks and target schedules. Layer weights then control the vector field; updating earlier layers may also change later keys. The fixed-operator proposition applies to the stated single-layer setting, not automatically to this coupled extension. Causal layer weights can supply an initial policy, but do not determine the optimal behavioral trade-off.

4 A PRESERVATION BARRIER WITH A TESTABLE MEANING

Let d_{raw} be the full-budget one-pass delta from W_{in} . The scaled baseline is $\gamma_{\text{ray}}(q) = W_{\text{in}} + qd_{\text{raw}}$ with a declared range $q \in [0, q_{\text{max}}]$. For a continuous path $\gamma : [0, 1] \rightarrow \mathbb{R}^{d \times p}$, define its peak preservation loss

$$M_D(\gamma) = \max_{s \in [0, 1]} D(\gamma(s)). \quad (10)$$

Fix a compute bound and a protected old-edit set with ceilings c_j . Let Γ be a declared family of paths beginning at W_{in} and satisfying the old-edit constraints along the path. Define

$$B_\Gamma(\varepsilon) = \inf_{\substack{\gamma \in \Gamma \\ \gamma(1) \in \mathcal{G}_\varepsilon}} M_D(\gamma), \quad \inf \emptyset = +\infty. \quad (11)$$

For the ray family this is the infimum, over successful scales q , of $\max_{0 \leq v \leq q} D(W_{\text{in}} + vd_{\text{raw}})$. For the controlled family, connect the successive writes in Equation (6) by their actual line segments; target-only phases leave W unchanged.

Definition 1 (Barrier bypass relative to a baseline family). *For a preservation budget d_{max} , a bypass occurs if*

$$B_{\Gamma_{\text{ctrl}}}(\varepsilon) \leq d_{\text{max}} < B_{\Gamma_{\text{ray}}}(\varepsilon). \quad (12)$$

Explicitly include the ray option in Γ_{ctrl} if a nesting claim is intended. The definitions concern specified families and budgets; they do not establish a global topological obstruction. Infima over a finite menu and maxima over sampled segment points provide empirical approximations. A sampled maximum is a lower bound on the continuous maximum, so it cannot by itself certify that a path stays below d_{max} . Temporary increases in current edit loss are allowed; demanding strict descent would further restrict the family.

A lower peak alone is insufficient. A one-shot algorithm can jump across a high-loss interpolation region. Moreover, identical final weights imply identical model predictions, regardless of how those weights were obtained. Report the endpoint frontier separately:

$$F_\Gamma(\varepsilon) = \inf_{\substack{\gamma \in \Gamma \\ \gamma(1) \in \mathcal{G}_\varepsilon}} D(\gamma(1)). \quad (13)$$

Practical evidence requires better endpoints at matched editing requirements, old-edit retention, evaluation-only generalization/locality, and acceptable cost. A lower peak supports a route interpretation only in addition to those outcomes. If final weights agree but future editing differs, inspect the editor’s history and caches; neural predictions do not remember the path.

Which barrier is meant? Three objects must stay distinct: a high actual preservation loss along a route (Equation (11)); a logarithmic constraint penalty used to select proposals (below); and a nonconvex obstruction in an objective’s landscape. None implies the other without evidence. Mode-connectivity work and ODE-M supply relevant route precedents (Garipov et al., 2018; Lin and Kang, 2026); they do not prove a barrier exists for the reported AlphaEdit run.

5 SELECTING TARGET STEPS AND WRITE SCALES

The ODE is a representation, not a schedule optimizer. A finite control problem is

$$\begin{aligned} \min_{N, \{m_k, \eta_k\}} \quad & D(W_N) + \xi M_D(\gamma_u) \\ \text{subject to} \quad & \text{Equation (6), } W_N \in \mathcal{G}_\varepsilon, \quad \gamma_u \in \Gamma, \\ & \text{cost}(u, \text{candidate search, probes}) \leq B_{\text{comp}}. \end{aligned} \quad (14)$$

Cost includes rejected branches, target gradients, factorizations, writes, teacher evaluations, coreset construction, and loss probes. A rough term $\sum_k (c_z m_k + c_w)$ omits much of this cost. Report measured wall time and forward/backward token counts; ODE time is not compute.

A first controller. Use a small menu of target checkpoints and write scales, evaluate the materialized candidates on control data, and retain a small beam for a second pass. Snapshotting checkpoints along one target run can share work, provided the optimizer and initialization match the corresponding shorter runs. Restore the parent model and editor state before every branch. Select the best successful final candidate under Equation (14); return the incoming state and mark the request unaccepted if none meets the declared criteria. Greedy first-stage efficacy can discard a useful route, so keep more than one branch and report the search restriction. No guarantee of finding the best admissible schedule follows from a two-stage beam.

Optional feedback through the write. Define the provisional model and a target objective

$$\begin{aligned} W^+(W, Z; \eta) &= W + \eta \mathcal{A}(W, Z; C), \\ J_{\text{route}}(W, Z; \eta) &= E(W^+) + \mu \mathcal{B}_{d_{\max}}(W^+) + \frac{\rho}{2} \|Z - H(W)\|_F^2, \\ \mathcal{B}_{d_{\max}}(V) &= -\log(1 - D(V)/d_{\max}), \quad D(V) < d_{\max}. \end{aligned} \quad (15)$$

The logarithm is defined only within its domain; infeasible candidates need a smaller step or a different proposal. Differentiating through the write introduces direct post-edit feedback and is closely related to MetaKE (Liu et al., 2026b). It is an extension beyond merely changing the schedule of the baseline target optimizer. With Equation (5), the loss gradient through the candidate write is multiplied by $\eta \mathcal{A}_C^T$; there is no need to differentiate a matrix inverse with respect to Z when $\mathcal{A}_C$ is fixed.

An early-stop diagnostic is the realization gap

$$G(W, Z; \eta) = E(W^+(W, Z; \eta)) - E_{\text{hook}}(W, Z), \quad (16)$$

using identical prompts, target tokens, and reductions in both terms. A decreasing hook loss with worsening candidate performance is a reason to stop or revise a target. The gap itself is not a topological barrier, and a short target optimization does not necessarily yield a small write.

Optional continuous guard. For a feasible initial state, a differentiable D , and an absolutely continuous trajectory, the control-barrier inequality

$$\langle \nabla D(W), \dot{W} \rangle \leq \kappa [d_{\max} - D(W)], \quad \kappa \geq 0, \quad (17)$$

implies $D(W(s)) \leq d_{\max}$ if it holds almost everywhere. Indeed, $h = d_{\max} - D$ satisfies $\dot{h} + \kappa h \geq 0$, whence $h(s) \geq e^{-\kappa s} h(0) \geq 0$. This is conditional invariance, not convergence to editing success; see control-barrier-function theory (Ames et al., 2019). At the boundary, scaling a single harmful direction may only permit zero speed. A different target direction must be available to continue. An arbitrary weight-space gradient correction enlarges the update family unless restricted to $\{BA_C\}$. Finite Euler writes, rounded weights, and approximate coreset gradients do not inherit this guarantee automatically.

6 SELECTING A CORESET FOR PRESERVATION LOSS

We propose a *weighted representative core selected by finite update responses*, plus separate vulnerable-example checks. This adapts weighted medoid selection and interference-based retrieval to the actual losses used by the controller. It is a design to evaluate, not a claim of a new general coreset principle. The roles and data boundaries matter as much as the score.

6.1 THREE DATA ROLES AND A SEPARATE ACCEPTED-EDIT LEDGER

Let $\mathcal{B} = \{x_i\}_{i=1}^M$ be a generic control bank, with fixed original-model teacher distributions. For a chunk with L_i scored tokens, let $\ell_i(W)$ be its mean full-vocabulary KL. Define

$$D_{\mathcal{B}}(W) = \sum_{i=1}^M \pi_i \ell_i(W), \quad \pi_i \geq 0, \quad \sum_i \pi_i = 1. \quad (18)$$

Choose $\pi_i = L_i / \sum_j L_j$ for a token-weighted objective, or declare different source/stratum masses in advance. This empirical bank objective approximates Equation (2); the guarantee below is about the bank, not the whole text distribution.

1. The **representative core** estimates the bank’s weighted mean. It has explicit cluster-mass weights, not one vote per selected chunk.
2. A **vulnerability set** checks examples with large predicted or measured damage. Its oversampling is intentional and must not silently redefine the average preservation loss.
3. A **witness panel** checks coreset fidelity on fresh control chunks. It can trigger a refresh and therefore is controller data. A separately locked reporting panel is never used for selection, thresholds, early stopping, or schedule choice.

An accepted-edit ledger is separate from all three. Its labels are accepted desired answers, not original-model predictions. For limited replay, combine a declared age/relation-stratified reservoir with currently vulnerable old edits and rotate coverage. Check losses individually on the selected $\mathcal{J}_t$. Sampled retention does not guarantee all 10,000 previous facts. A stored full ledger permits periodic full checks, which must also be budgeted. Resolve contradictory accepted requests by an explicit task rule; otherwise simultaneous retention constraints can be infeasible.

6.2 WHY FINITE RESPONSES RATHER THAN KL GRADIENTS ALONE?

At the original model, every exact teacher KL satisfies

$$\ell_i(W_{\text{orig}}) = 0, \quad \nabla \ell_i(W_{\text{orig}}) = 0, \quad \ell_i(W_{\text{orig}} + d) = \frac{1}{2} \langle d, F_i d \rangle + o(\|d\|_F^2), \quad (19)$$

under ordinary differentiability assumptions, with the local output Fisher form $F_i \succeq 0$. Thus matching these first-order gradients at initialization cannot rank which chunks will be damaged. At later states gradients can help, but may still miss finite-write curvature.

At a refresh state W_c , generate a small collection of *allowed candidate displacements* $d^{(q)}$ using current or already-seen requests: short/full target checkpoints, a few write scales, and optionally midpoints of their write segments. Include $d^{(0)} = 0$. Evaluate the response signature

$$\phi_i = (\ell_i(W_c + d^{(0)}), \dots, \ell_i(W_c + d^{(Q)})) \in \mathbb{R}^{Q+1}. \quad (20)$$

Use the same raw loss units across coordinates for the bound below. If coordinates are standardized, undo that scaling when bounding the original losses. These signatures capture finite effects downstream of layer 4 and remain informative when first-order KL gradients vanish. Future request labels and evaluation queries are unavailable to selection.

6.3 WEIGHTED MEDOIDS AND AN EXPLICIT APPROXIMATION BOUND

Select m representative chunks by approximately solving

$$S \in \arg \min_{|S| \leq m} \Phi(S), \quad \Phi(S) = \sum_i \pi_i \min_{j \in S} \|\phi_i - \phi_j\|_{\infty}. \quad (21)$$

A concrete greedy implementation starts with the best single medoid, then adds the chunk with the largest decrease in Φ until the budget is met; optional swaps can improve the result. Domain quotas can restrict assignments within predeclared strata. This is a facility-location/weighted-medoid construction, related to CRAIG (Mirzasoileiman et al., 2020), applied here to finite losses rather than gradient representations. No exact optimization of Equation (21) is required for the following bound.

Assign each bank chunk to a nearest chosen medoid $a(i) \in S$ and transfer its mass:

$$w_j = \sum_{i:a(i)=j} \pi_i, \quad \hat{D}_S(W) = \sum_{j \in S} w_j \ell_j(W), \quad \sum_{j \in S} w_j = 1. \quad (22)$$

Proposition 3 (Finite-candidate loss approximation). *For the signatures in Equation (20), let $\varepsilon_{\text{core}} = \sum_i \pi_i \|\phi_i - \phi_{a(i)}\|_\infty$. Then*

$$\max_{0 \leq q \leq Q} \left| D_B(W_c + d^{(q)}) - \hat{D}_S(W_c + d^{(q)}) \right| \leq \varepsilon_{\text{core}}. \quad (23)$$

Proof. The difference at coordinate q equals $\sum_i \pi_i [\phi_{i,q} - \phi_{a(i),q}]$. Apply the triangle inequality and bound each coordinate difference by its signature’s infinity norm. $\square$

The same bound controls the difference between the bank and coreset maxima over this finite candidate set. For a new state W within distance r of a probed state, if every ℓ_i is L -Lipschitz on the relevant region, the triangle inequality gives

$$|D_B(W) - \hat{D}_S(W)| \leq \varepsilon_{\text{core}} + 2Lr. \quad (24)$$

This is a conditional extension, not a usable certificate when L or trajectory coverage is unknown. In particular, loss-value matching supplies no gradient approximation guarantee for Equation (17). Matching directional derivatives or curvature would require additional features and its own error analysis.

What can be checked honestly? For a represented candidate, $\hat{D}_S + \varepsilon_{\text{core}} \leq d_{\text{max}}$ implies the bank bound. For future candidates, a witness discrepancy is a diagnostic, not a certified replacement for the unknown approximation error. An empirical controller can screen with the core and witness; a strict finite-bank acceptance arm evaluates the final candidate on the entire bank. Neither arm certifies unprobed intermediate maxima or distribution-wide preservation. Keep these arms distinct in results.

6.4 VULNERABILITY CHECKS AND REFRESH RULES

For generic chunk i , use an interference score such as

$$v_i = \max_q [\ell_i(W_c + d^{(q)}) - \ell_i(W_c)]_+. \quad (25)$$

Keep the top r_v chunks, with diversity or stratum limits to avoid selecting near duplicates. Also include chunks already near their declared absolute loss ceilings. Equation (25) is inspired by MIR’s virtual-update interference criterion (Aljundi et al., 2019b). At each candidate, check $\ell_i(W) \leq \delta_i$ for selected vulnerable chunks. These constraints are separate from $\hat{D}_S$; top-loss sampling without mass correction would estimate a different population. If a tail objective is desired, define and report it as such instead of calling it average KL.

Use the analogous increase in R_j^{old} for old-edit vulnerability, combined with the reservoir/coverage component described above. Do not collapse old desired-label loss and generic teacher KL into one unlabeled average. GSS, GCR, and influence-based replay provide alternative selection baselines (Aljundi et al., 2019a; Tiwari et al., 2022; Sun et al., 2023).

Freeze the bank definition, core, weights, ceilings, and vulnerability set during an inner path so its apparent barrier cannot fall merely because the measuring set changed. Refresh between request episodes, at a declared interval or after a predeclared witness-error trigger. Reevaluate the current state under the refreshed core and retain full-bank/reporting measurements on a consistent population. If an inner episode needs a refresh, restart its comparison from the same parent state and rescore all retained branches. Any witness used to adapt the controller is no longer held out; reserve new chunks or maintain a separate reporting split.

6.5 A BUDGETED INITIAL CONFIGURATION

The following is an implementation starting point, not tuned hyperparameters or an accuracy guarantee:

Component	Initial choice and purpose
Control bank	512 generic chunks, 128 scored tokens each; fixed source masses.
Representative core	64 weighted response medoids.
Vulnerability set	16 chunks selected by damage increase and absolute margins.
Witness	64 fresh control chunks per refresh window; separate reporting panel.
Old-edit checks	64 stratified reservoir requests plus up to 32 vulnerable requests.
Response proposals	Target checkpoints $m \in \{5, 25\}$, scales $\eta \in \{0.25, 0.5, 1\}$, plus the unmodified state.
Schedule search	Two passes, beam width 2; second pass uses 25 target steps and the declared scale menu.
Refresh	Every 10 request batches, or a predeclared witness discrepancy trigger.

All numbers must be adjusted using development streams, then fixed before the reported 10k-edit evaluation. Include segment probes when measuring barriers, even if endpoint-only screening is used for ordinary editing.

Constructing signatures costs roughly $(Q + 1) \sum_i L_i$ edited-model forward tokens, plus teacher work and proposal generation. Greedy distance selection can cost $O(mM^2Q)$ in a simple implementation; cached nearest distances and candidate subsampling can reduce constants. At the refresh episode the full-bank candidate losses are already known, so ranking those very same candidates with the core saves no forward work. The intended saving is reuse for subsequent probes or differentiable candidate optimization; reuse loses the finite-set guarantee unless coverage is established. Measure amortized cost and compare a uniform/stratified core at the same wall time. If frequent refreshes erase the saving, the simpler sampler is preferable.

Stream full-vocabulary teacher KL in chunks or cache it only when storage permits. Large-vocabulary teacher logits can dominate memory. Top- k renormalization changes the loss and invalidates an exact full-KL claim unless its omitted-mass error is separately controlled.

7 RELATED WORK AND CITATION TRAILS

We inspected primary papers and bibliographies, then expanded selected seeds through forward-citation queries. Here $A \rightarrow B$ means A *cites* B . OpenAlex was used for discovery; the substantive edges below were checked in primary bibliographies. The audit date is 14 September 2026. The companion `research/multi_step_ode_related_work_audit.md` records query identifiers, evidence locations, version issues, and exclusions. This is a targeted citation neighborhood, not an exhaustive review or a novelty proof.

7.1 LOCATION, LAYER ALLOCATION, AND UPDATE STRENGTH

ROME and MEMIT (Meng et al., 2022; 2023) establish the activation-target and associative-memory editing framework. AlphaEdit (Fang et al., 2025) adds projection-based preservation. Hase et al. (2023) directly question whether localization results predict editing performance. BLUE (Li et al., 2025) focuses on residual distribution and boundary-layer updates; its comparison is more specific than deleting arbitrary layers while holding every other implementation choice fixed. These works support separating causal attribution from the conditioning and fidelity of a write.

PMET (Li et al., 2024) changes residual allocation as well as target optimization; MEND (Mitchell et al., 2022) learns update transformations and step sizes. CAKE (Liu and Zhang, 2026) allocates residuals with causal-guided layer weights. Given a fixed linear write map and residual, weighting that residual scales the corresponding delta. Across an actual multilayer sequence, earlier writes can change later residuals and keys, so this local equivalence is not necessarily a global rescaling of an unchanged baseline delta list. Our single-layer study removes allocation across layers and varies effort *across refinement passes*. The two controls can eventually be combined.

7.2 REPEATED EDITING AND TARGET OPTIMIZATION

The direct iterative precedent. Baghel et al. (2025) repeat both target optimization and weight spreading on the updated model. Their Appendix D also tests repeated spreading with a fixed target. This directly precedes the alternating loop; two calls alone are not a new method. Their Llama-3.1 AlphaEdit comparison from one to three iterations raises efficacy and generalization while reducing specificity, so their result does not establish the reported three-metric improvement for a short/full layer-4 schedule. The distinction to test is budget allocation, write scaling, preservation feedback, and long-sequence retention. Their bibliography cites ROME, MEMIT, PMET, and AlphaEdit. Forward expansion found Zhang et al.’s 2026 Nature Machine Intelligence Perspective (Zhang et al., 2026), whose reference 63 cites this paper; that is a discussion citation, not an independent replication.

NSE (Jiang et al., 2025a) also uses repeated writes to address insufficient editing, with neuron selection and remaining-error checks. It constructs targets using original model weights, which differs from refreshing the target on each currently edited model. It is a relevant residual-fitting comparator, not evidence that all iterative editors implement Equation (6) identically.

Early stopping and long-horizon regularization. Gupta et al. (2025) study excessive target optimization and propose Most-Probable Early Stopping with edited-weight norm regularization in ENCORE. NAS (Liu et al., 2026a) rescales solved value vectors to original-model reference norms. NAS cites the regularization work, furnishing a verified forward trail. Value normalization, residual scaling, and early stopping intervene at different points; none should be treated as interchangeable by name. They motivate tracking target norms, delta norms, and realization errors alongside the short/full schedule.

Optimizing targets through executable updates. MetaKE (Liu et al., 2026b) refines targets using virtual post-edit losses and a structural proxy for downstream writes, then executes the edit. Its latest inspected version is arXiv v3, which also cites LyapLock, AdaEdit, and AnyEdit. The target-through-write objective in Equation (15) therefore has direct prior art. Our distinct question is how *real intermediate writes and asymmetric target budgets* compare with that virtual look-ahead under matched resources and original-reference preservation. The exact-DOI OpenAlex query returned no forward citations for MetaKE’s record on the audit date; this is limited index coverage, not a claim that it is uncited.

Backward expansion to AdaEdit (Li and Chu, 2025) identifies adaptive selection and disentanglement of editing parameters; AnyEdit (Jiang et al., 2025b) breaks long desired outputs into autoregressive editing stages. AnyEdit’s “efficacy barrier” concerns realizing long-output edits, not our maximum preservation loss along a weight path. These papers are relevant to adaptive effort and sequential targets, but do not validate the proposed bypass.

7.3 CURVED PATHS, PRESERVATION CONSTRAINTS, AND CONTINUOUS DYNAMICS

Garipov et al. (2018) construct low-loss connections between trained solutions. Such connectivity motivates looking beyond a straight interpolation, but does not imply the current and desired edited models are connected within an AlphaEdit write family.

ODE-M (Lin and Kang, 2026) applies continuous weight trajectories and barrier-aware gradient rectification to continual model merging. Its bibliography points back to mode connectivity, permutation-related loss landscapes, task arithmetic, and uncertainty-based gradient matching (Garipov et al., 2018; Entezari et al., 2022; Ilharco et al., 2023; Daheim et al., 2024). This is a close conceptual predecessor to the ODE/barrier framing. Merging fixed task models differs from allocating alternating target learning and writes for an edit. The queried ODE-M records returned no indexed forward links; recent papers and duplicate records make that result inconclusive about influence.

LyapLock (Wang et al., 2025) is closer in application: it uses virtual queues and Lyapunov optimization for sequential editing with a long-run average preservation constraint. Its formulation measures preservation through key/value reconstruction loss. It cites Neely’s stochastic-network optimization framework (Neely, 2010); MetaKE v3 cites LyapLock. A time-average constraint does not imply a pointwise ceiling on original-model output KL along every inner write. Our optional guard borrows the latter kind of invariance condition from control-barrier-function theory (Ames

et al., 2019). Neither a generic preservation constraint nor control-theoretic terminology is a new contribution here.

7.4 CORESETS: WHAT TO APPROXIMATE, AND WHEN TO REFRESH

Weighted mean and gradient approximation. CRAIG (Mirzsoleiman et al., 2020) selects weighted representatives for efficient learning through gradient approximation and facility-location structure. Grad-Match (Killamsetty et al., 2021) uses weighted gradient matching with an orthogonal matching pursuit construction. GCR (Tiwari et al., 2022) applies gradient coreset selection to replay in continual learning. Its bibliography explicitly cites CRAIG, Grad-Match, GSS, MIR, and bilevel coresets. This gives both a backward map for GCR and a forward trail from the earlier subset methods. Our response core matches finite *loss values* on a candidate menu; it does not inherit their gradient or training-convergence guarantees.

Curvature and locality of a core. AdaCore (Pooladzandi et al., 2022) uses curvature information in subset selection and cites CRAIG and Grad-Match. CREST (Yang et al., 2023) develops coresets for local training regions and cites all three. This verified chain motivates adding finite second-order information and refreshing after movement; it does not justify transporting one fitted core across an arbitrary 10k-edit stream. Equation (19) gives a particularly direct reason to avoid gradient-only selection at the original teacher.

Coverage and threatened examples. GSS (Aljundi et al., 2019a) selects memory examples using gradient diversity, relating selection to the constraints represented by replay data; its references include GEM and A-GEM (Lopez-Paz and Ranzato, 2017; Chaudhry et al., 2019). MIR (Aljundi et al., 2019b) retrieves examples whose losses increase under a prospective update. These ideas have distinct roles in our design: representative coverage estimates a declared mean, while interference identifies examples worth explicit protection. A worst-case-biased retrieval set should not be reported as an unweighted estimate of the original population.

Selection affects subsequent selection. Borsos et al. (2020) formulate coreset selection as bilevel optimization with an outer data objective; this is an alternative to the response-matching proxy, at additional optimization cost. Forward searches from Grad-Match and GCR found Sun et al. (2023). Its references explicitly include both, as well as GSS and bilevel coresets. It studies second-order interactions between successive selections, warning that a buffer optimized only for today’s criterion can bias tomorrow’s candidates. This supports maintaining representative coverage and independently measuring drift, rather than repeatedly retaining only today’s most threatened chunks.

7.5 THE RESULTING CONTRIBUTION BOUNDARY

The proposed contribution would be a useful, resource-accounted controller for target horizons and writes, together with evidence explaining its behavior in sequential editing. The differentiating components to evaluate are an asymmetric real-write schedule at a fixed layer; a barrier definition paired with an endpoint frontier; and a practical response core with explicit mass weights, threat checks, and a finite-candidate error statement. Iteration, virtual-write target optimization, ODE paths, barrier guards, weighted medoids, and interference retrieval each have identifiable prior art. A successful combination and a measured mechanism remain to be demonstrated.

8 EXPERIMENTS THAT DISTINGUISH THE EXPLANATIONS

First replay identical incoming model/editor states. Then run paired full 10k streams with the same request order and several seeds. Preserve the reported batch size 100 and layer-4 target/write locations; record the exact backbone checkpoint and indexing convention. Compare:

1. Full one-pass AlphaEdit, and its best scanned write scale over a declared range, including the search cost.
2. One target run with the same total target-gradient budget as the two-pass method, and two full-budget passes.

3. Short/full versus full/short schedules with the same total target steps and write-scale menu; record both deployed and rejected candidates.
4. Frozen-target repeated residual fitting, to isolate Equation (9) from refreshed targets.
5. Direct target optimization through the fixed write map, with a MetaKE-style objective, versus actual interleaved writes.
6. ENCORE-style stopping/norm control and NAS, where compatible, to test whether the gains are explained by simpler regularization.
7. Once-per-request history registration versus the exact repeated-call history behavior of the motivating run; fresh targets versus cache reuse.

Use both target-step-matched and wall-time-matched comparisons. If a schedule search has a larger development budget, disclose it separately rather than charging only for its winning branch.

For coreset ablations, compare full-bank decisions, uniform or stratified sampling, gradient matching, response medoids, and response medoids plus vulnerability checks. Sweep core size and refresh interval at matched total cost. Report candidate-loss approximation error, peak-loss error on dense diagnostic probes, false accepts/rejects relative to the bank, witness and reporting-panel drift, and old-edit coverage by age/relation. Loss matching on selected candidates can still overfit the bank; the locked panel tests this failure directly.

Record hook loss, actual edit loss, generic KL, each selected old-edit loss, target and update norms, target-versus-realized activation error, Equation (16), and the cosine between successive deltas. Verify key equality, projector residuals, cache state, and history changes. Thresholds and original-model references stay fixed during a comparison. Retention and generalization are separate outcomes from current request NLL.

A real-model landscape diagnostic. Let $e_1 = d_{\text{raw}} / \|d_{\text{raw}}\|_F$, and let e_2 be the normalized component of $W_N - W_{\text{in}}$ orthogonal to e_1 . If this component is negligible, report a nearly collinear endpoint instead of inventing a second direction. Evaluate the actual LLM on a grid $W_{\text{in}} + ae_1 + be_2$, with preservation contours and individual editing-success regions. Overlay the ray and the controlled route. A projected route may leave this plane, so score its actual states separately. Adaptive segment refinement can reveal missed peaks; a finite grid still does not prove a continuous barrier bound.

Decision rules for the scientific claim. If matched-budget one-pass editing removes the gain, additional optimization may explain it. If fixed-target repetition or tuned scaling matches the gain, residual fitting or regularization may suffice. If endpoints improve without a measured barrier difference, retain the optimization result and drop the bypass claim. Evidence for the proposed mechanism requires a better matched-success endpoint frontier, lower measured route peaks relative to a strong scaled baseline, and persistence through long paired streams on independent reporting data. No result of that kind is claimed in this note.

REFERENCES

- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and Editing Factual Associations in GPT. *NeurIPS*, 2022. <https://arxiv.org/abs/2202.05262>.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-Editing Memory in a Transformer. *ICLR*, 2023. <https://arxiv.org/abs/2210.07229>.
- Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models. *NeurIPS*, 2023. <https://arxiv.org/abs/2301.04213>.
- Junfeng Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. *ICLR*, 2025. <https://arxiv.org/abs/2410.02355>.
- AlphaEdit authors. Official implementation: write and target-optimization code. Accessed 14 September 2026. <https://github.com/jianghoucheng/AlphaEdit/tree/main/AlphaEdit>.

- Xiaopeng Li, Shanwen Wang, Shasha Li, Shezheng Song, Bin Ji, Jun Ma, and Jie Yu. Rethinking Residual Distribution in Locate-then-Edit Model Editing. *NeurIPS*, 2025. Inspected arXiv v3. <https://arxiv.org/abs/2502.03748v3>.
- Xiaopeng Li et al. PMET: Precise Model Editing in a Transformer. *AAAI*, 2024. <https://arxiv.org/abs/2308.08742>.
- Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast Model Editing at Scale. *ICLR*, 2022. <https://arxiv.org/abs/2110.11309>.
- Shuxin Liu and Jianhao Zhang. CAKE: Causal-Guided Adaptive Knowledge Editing for LLMs. *ACL*, 2026. <https://aclanthology.org/2026.acl-long.918/>.
- Bhiman Kumar Baghel, Emma Jordan, Zheyuan Ryan Shi, and Xiang Lorraine Li. Resolving UnderEdit & OverEdit with Iterative & Neighbor-Assisted Model Editing. *Findings of EMNLP*, 2025. <https://aclanthology.org/2025.findings-emnlp.798/>.
- Ningyu Zhang et al. Towards principled knowledge editing methods for large language model reasoning. *Nature Machine Intelligence*, 8:1189–1200, 2026. <https://doi.org/10.1038/s42256-026-01276-y>.
- Houcheng Jiang et al. Neuron-Level Sequential Editing for Large Language Models. *ACL*, 2025. <https://aclanthology.org/2025.acl-long.815/>.
- Akshat Gupta et al. Lifelong Knowledge Editing Requires Better Regularization. *Findings of EMNLP*, 2025. <https://aclanthology.org/2025.findings-emnlp.1234/>.
- Mingda Liu, Zhenghan Zhu, Ze’an Miao, and Katsuki Fujisawa. Norm Anchors Make Model Edits Last. arXiv:2602.02543v3, 2026. <https://arxiv.org/html/2602.02543v3>.
- Shuxin Liu, Di Gao, and Ou Wu. MetaKE: Meta-Learning for Knowledge Editing Toward a Better Accuracy-Editability Trade-off. arXiv:2603.12677v3, 2026. HTML heading: Meta-learning Aligned Knowledge Editing via Bi-level Optimization. <https://arxiv.org/abs/2603.12677v3>.
- Qi Li and Xiaowen Chu. AdaEdit: Advancing Continuous Knowledge Editing For Large Language Models. *ACL*, 2025. <https://aclanthology.org/2025.acl-long.208/>.
- Houcheng Jiang et al. AnyEdit: Edit Any Knowledge Encoded in Language Models. *ICML*, 2025. <https://proceedings.mlr.press/v267/jiang25b.html>.
- Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs. *NeurIPS*, 2018. <https://arxiv.org/abs/1802.10026>.
- Lihong Lin and Haidong Kang. Unlocking the Potential of Continual Model Merging: An ODE Perspective. arXiv:2605.19409v3, 2026. <https://arxiv.org/html/2605.19409v3>.
- Rahim Entezari, Hanie Sedghi, Olga Saukh, and Behnam Neyshabur. The Role of Permutation Invariance in Linear Mode Connectivity of Neural Networks. *ICLR*, 2022. <https://arxiv.org/abs/2110.06296>.
- Gabriel Ilharco et al. Editing Models with Task Arithmetic. *ICLR*, 2023. <https://arxiv.org/abs/2212.04089>.
- Nico Daheim, Thomas Möllenhoff, Edoardo Maria Ponti, Iryna Gurevych, and Mohammad Emtiyaz Khan. Model Merging by Uncertainty-Based Gradient Matching. *ICLR*, 2024. <https://arxiv.org/abs/2310.12808>.
- Peng Wang, Biyu Zhou, Xuehai Tang, Jizhong Han, and Songlin Hu. LyapLock: Bounded Knowledge Preservation in Sequential Large Language Model Editing. *EMNLP*, 2025. <https://aclanthology.org/2025.emnlp-main.327/>.
- Michael J. Neely. Stochastic Network Optimization with Application to Communication and Queueing Systems. Morgan & Claypool, 2010. [Publisher DOI](#).

- Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control Barrier Functions: Theory and Applications. *ECC*, 2019. <https://arxiv.org/abs/1903.11199>.
- Baharan Mirzasoileiman, Jeff Bilmes, and Jure Leskovec. Coresets for Data-efficient Training of Machine Learning Models. *ICML*, 2020. <https://proceedings.mlr.press/v119/mirzasoileiman20a.html>.
- Krishnateja Killamsetty, Durga S, Ganesh Ramakrishnan, Abir De, and Rishabh Iyer. GRAD-MATCH: Gradient Matching Based Data Subset Selection for Efficient Deep Model Training. *ICML*, 2021. <https://proceedings.mlr.press/v139/killamsetty21a.html>.
- Rishabh Tiwari, Krishnateja Killamsetty, Rishabh Iyer, and Pradeep Shenoy. GCR: Gradient Coreset Based Replay Buffer Selection for Continual Learning. *CVPR*, 2022. <https://arxiv.org/abs/2111.11210>.
- Omead Pooladzandi, David Davini, and Baharan Mirzasoileiman. Adaptive Second Order Coresets for Data Efficient Machine Learning. *ICML*, 2022. <https://proceedings.mlr.press/v162/pooladzandi22a.html>.
- Yu Yang, Hao Kang, and Baharan Mirzasoileiman. Towards Sustainable Learning: Coresets for Data-efficient Deep Learning. *ICML*, 2023. <https://proceedings.mlr.press/v202/yang23g.html>.
- Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient Based Sample Selection for Online Continual Learning. *NeurIPS*, 2019. <https://arxiv.org/abs/1903.08671>.
- Rahaf Aljundi, Lucas Caccia, Eugene Belilovsky, Massimo Caccia, Min Lin, Laurent Charlin, and Tinne Tuytelaars. Online Continual Learning with Maximal Interfered Retrieval. *NeurIPS*, 2019. <https://arxiv.org/abs/1908.04742>.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient Episodic Memory for Continual Learning. *NeurIPS*, 2017. <https://arxiv.org/abs/1706.08840>.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient Lifelong Learning with A-GEM. *ICLR*, 2019. <https://arxiv.org/abs/1812.00420>.
- Zalán Borsos, Mojmír Mutný, and Andreas Krause. Coresets via Bilevel Optimization for Continual Learning and Streaming. *NeurIPS*, 2020. <https://arxiv.org/abs/2006.03875>.
- Zhicheng Sun, Yadong Mu, and Gang Hua. Regularizing Second-Order Influences for Continual Learning. *CVPR*, 2023. <https://arxiv.org/abs/2304.10177>.